UNIVERSITY OF GHANA

DEPARTMENT OF COMPUTER SCIENCE

CSCD602 — ADVANCED SOFTWARE ENGINEERING

TECHNICAL DEBT PLAN

ICCTECH: A Web-Based IT Service Management and Helpdesk System

Student Name	Clement Asamoah
Student ID	22424193
Project	ICCTECH
Academic Year	First Semester, 2025/2026
Live Application	http://45.79.223.146:8080/index.php
Source Repository	https://github.com/Clemzy123/ICCTECH

1. Technical Debt Management Approach

Technical debt is treated as a visible engineering obligation rather than hidden unfinished work. Items are identified from source/configuration, deployment, testing/traceability, security, maintainability and operational-resilience review. Each record retains Debt → Cause → Impact → Priority → Proposed Resolution, plus classification, target and evidence.

1.1 Identification Sources

- Source-code and configuration review, including Docker and database configuration.
- Deployment architecture review of the Linode-hosted environment.
- Testing and requirements traceability review.
- Security-hardening review of authentication, transport and network exposure.
- Maintainability review of internal naming, automation and release traceability.
- Operational review of backup, monitoring, resilience and future scaling needs.

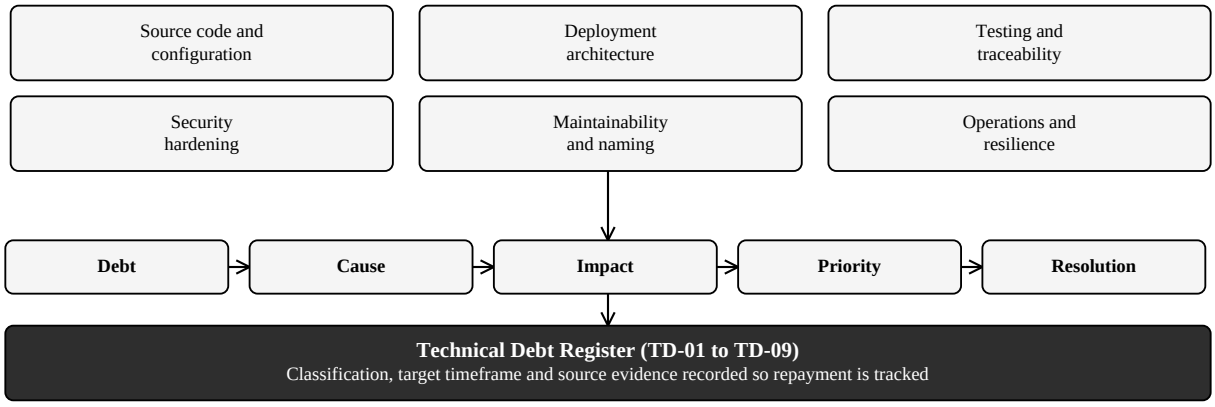

Figure 1. Technical debt identification and recording process

1.2 Priority and Classification

Classification	Meaning	Treatment
Critical / Immediate	Creates unacceptable security or release risk for continued production use.	Resolve before long-term public/production operation.
High / Scheduled	Does not prevent examination demonstration but materially affects security, reliability or regression confidence.	Schedule into production hardening or first maintenance release.
Medium / Managed	Acceptable temporarily under the examination scope but affects maintainability, resilience or future scalability.	Document, review each release and repay according to operational need.

Table 1. Technical debt classification model

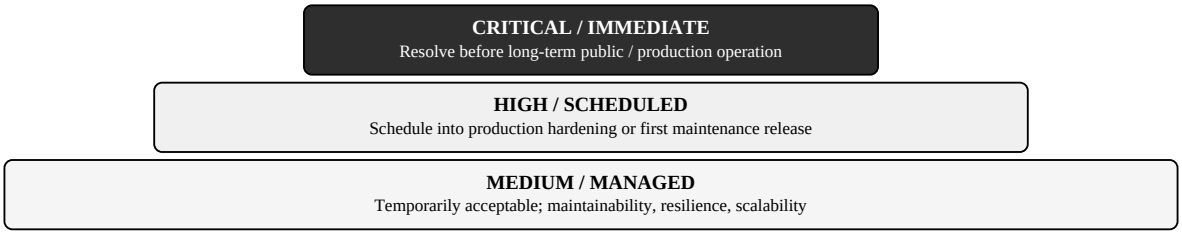

Figure 2. Technical debt classification model

2. Current Deployment Debt and Immediate Hardening

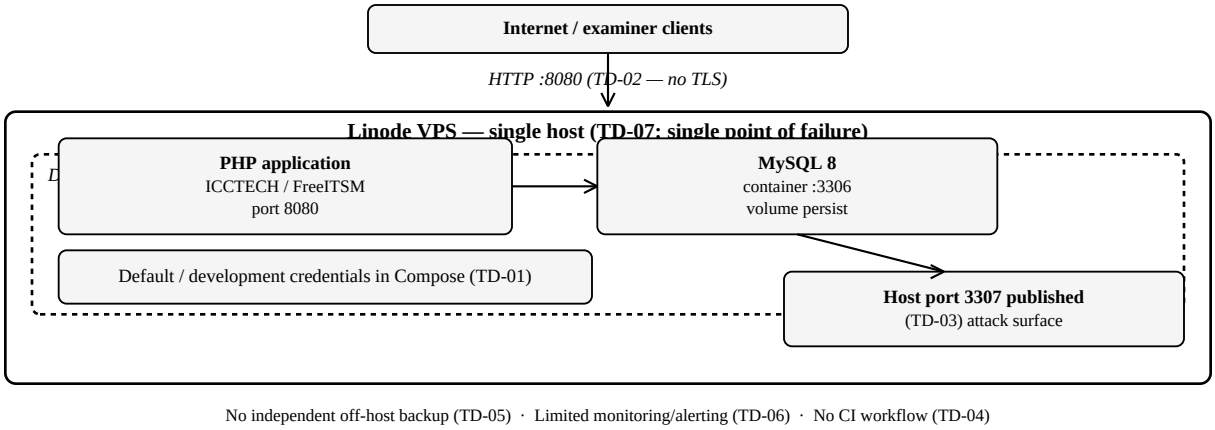

Figure 3. Current ICCTECH deployment architecture and associated debt (TD-01, TD-02, TD-03, TD-07; TD-04/05/06 noted)

TD-01 — Development/default credentials in Docker configuration

Debt	Development/default credentials remain in Docker configuration.
Cause	Rapid reproducible Docker setup.
Impact	Public reuse can expose credentials and enable unauthorised database access.
Priority	Critical
Classification	Immediate resolution
Proposed Resolution	Rotate credentials; move production secrets to protected environment or Docker secrets.
Target	Before continued public/production use
Source Evidence	Compose declares database credential variables.

TD-02 — HTTP rather than HTTPS

Debt	The live deployment currently uses HTTP.
Cause	The IP:8080 deployment was completed without TLS.
Impact	Authentication and application traffic is unencrypted in transit.
Priority	Critical
Classification	Immediate resolution
Proposed Resolution	Add a domain or reverse proxy, trusted TLS and HTTP-to-HTTPS redirect.
Target	Before long-term production use
Source Evidence	The live URL uses HTTP on port 8080.

TD-03 — Published MySQL host port

Debt	The Compose deployment publishes the MySQL host port.
Cause	Simplified development and direct database administration.
Impact	Published database access increases the attack surface.
Priority	High
Classification	Production hardening
Proposed Resolution	Remove the public mapping; use the internal Docker network, an SSH tunnel or a restricted firewall.
Target	Before hardened production use
Source Evidence	Compose maps host 3307 to MySQL 3306.

3. Regression, Recovery and Observability Debt

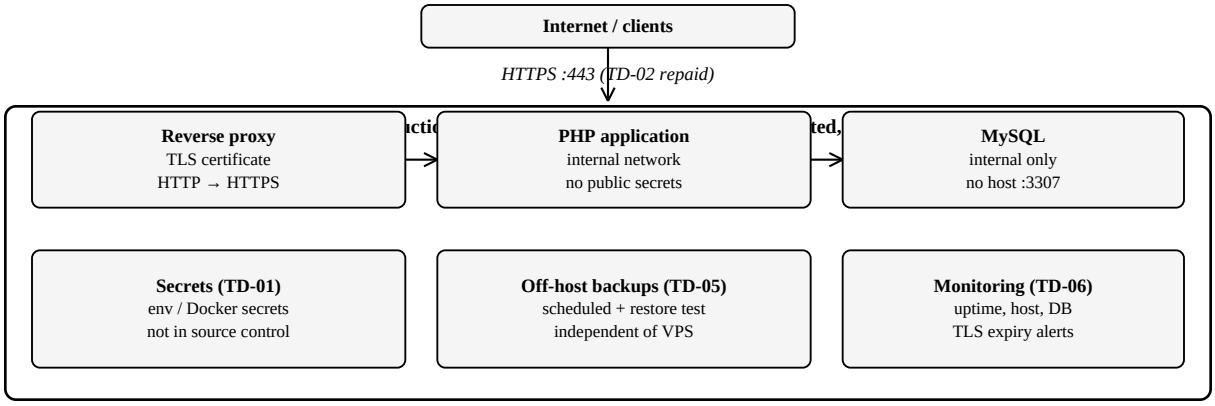

Figure 4. Target production-hardened architecture after repayment of TD-01, TD-02, TD-03, TD-05 and TD-06

TD-04 — Limited project-specific regression automation / CI

Debt	Requirements-mapped automated regression and CI are limited.
Cause	The 48-hour scope prioritised the core workflow; no mapped CI suite is demonstrated.
Impact	Future changes may create regressions detected late or only manually.
Priority	High
Classification	Scheduled for next release
Proposed Resolution	Automate mapped authentication, RBAC, ticket and persistence tests and add CI.
Target	First maintenance release
Source Evidence	tests/ exists; no CI workflow is demonstrated.

TD-05 — No automated independent off-host backup / restore

Debt	No demonstrated automated off-host backup and restore process.
Cause	Examination scope focused on persistence, not disaster-recovery automation.
Impact	Volumes do not protect against host loss, corruption or destructive change.
Priority	High
Classification	Scheduled for next release
Proposed Resolution	Schedule off-host MySQL backups with retention/protection and restore tests.
Target	First maintenance release
Source Evidence	Volumes exist; no off-host restore workflow is demonstrated.

TD-06 — Limited production monitoring / alerts

Debt	Production monitoring and automated alerting are limited.
Cause	Effort focused on functionality, testing and deployment, not observability.
Impact	Application, database, container or host failures may be noticed only after user reports.
Priority	Medium
Classification	Scheduled
Proposed Resolution	Add uptime, host/container/database metrics, alerts and TLS-expiry checks.
Target	Subsequent maintenance release
Source Evidence	No project production monitoring or alerting is demonstrated.

4. Resilience, Naming and Release-Traceability Debt

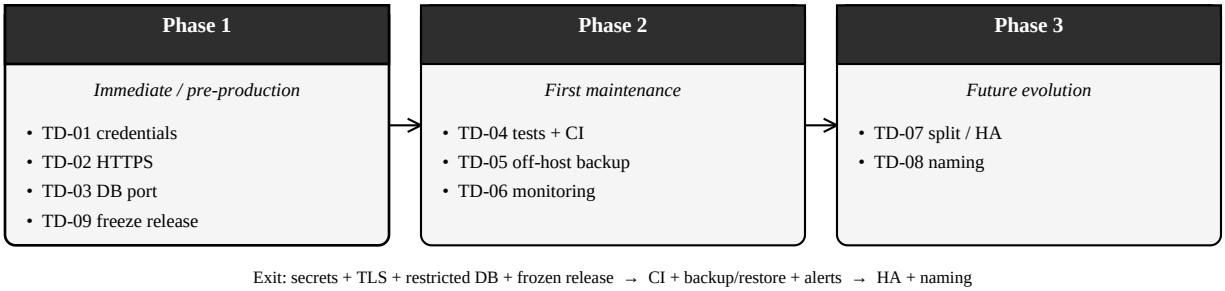

Figure 5. Technical debt repayment roadmap

Figure 5 presents the repayment sequence as a visual roadmap. Each phase must meet its exit condition before the next phase is treated as complete.

TD-07 — Single-host application and database

Debt	Application and database share one Linode host.
Cause	Single-server Docker was simplest for the examination.
Impact	Single point of failure; limited high availability and horizontal scaling.
Priority	Medium
Classification	Temporarily acceptable / future evolution
Proposed Resolution	When justified, separate workloads; add a managed or replicated database, health checks and load balancing.
Target	Future production evolution
Source Evidence	Application and MySQL run on one Linode host.

TD-08 — Legacy/upstream internal naming

Debt	Legacy/upstream identifiers remain in configuration and resources.
Cause	Broad 48-hour renaming would add regression risk.
Impact	Legacy names reduce maintainability and naming consistency.
Priority	Medium
Classification	Managed / scheduled refactoring
Proposed Resolution	Refactor incrementally with regression tests; preserve attribution and licences.
Target	Future refactoring releases
Source Evidence	FreeITSM identifiers remain in configuration and paths.

TD-09 — Documentation/evidence release drift

Debt	Documentation and evidence can drift from the deployed release.
Cause	Source, documents, screenshots and tests were produced rapidly in one window.
Impact	Revision mismatch weakens traceability and can confuse examination evidence.
Priority	Medium
Classification	Resolve before submission
Proposed Resolution	Freeze or tag the final build; verify evidence against it; avoid untracked changes.
Target	Before final Sakai submission
Source Evidence	Traceability depends on the final repository and deployed state.

5. Repayment Roadmap, Governance and Relationship to Testing

Repayment order. Immediate/pre-production: TD-01, TD-02, TD-03 and TD-09. First maintenance release: TD-04, TD-05 and TD-06. Future evolution: TD-07 and TD-08. A debt item is repaid only when the resolution is implemented and verified — for example, backup debt closes only after an independent backup is successfully restored.

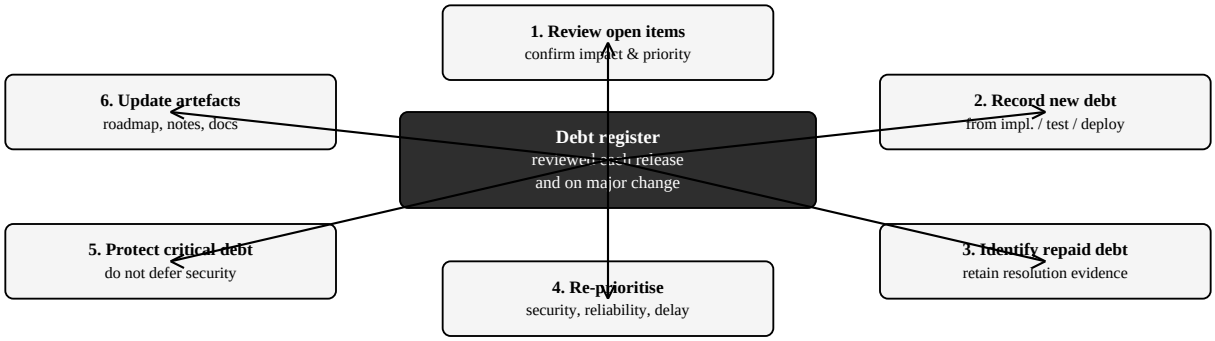

Figure 6. Review, governance and change-control cycle

The register is reviewed at each release or major architecture/security change. Critical security debt takes precedence over new features. High-priority reliability and test-automation debt is planned into the next release cycle. Medium-priority debt is ranked using user impact, operational risk, implementation effort and the likelihood that the debt will become more expensive to repay later.

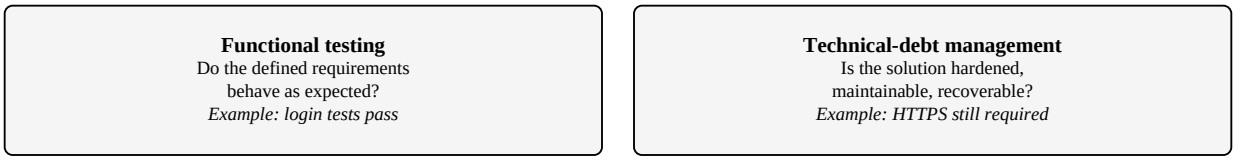

Figure 7. Functional testing versus technical-debt management

Functional acceptance and technical debt answer different questions: all selected tests may pass while hardening, recoverability, observability or scalability debt remains open. A login function may pass authentication tests while the deployment still requires HTTPS; ticket persistence may survive service restart while independent disaster-recovery backup remains future work (TD-05). Successful acceptance testing is not complete elimination of engineering risk.